

Review Report

Project: Fantasy Football League

Reviewers: It works on my wish

Section	Evaluation
1. Methodology Under Review	Tasarım, Olay Güdümlü (Event-driven) bir mimari üzerine kurulmuştur. Üç temel senaryo etrafında şekillenir: Fantazi takım oluşturma (Team Builder), Puan Hesaplama Sistemi (Strategy-based Scoring Engine) ve Global Liderlik Tablosunun (Leaderboard) yönetilmesi. Sistem, dışarıdan alınan periyodik veri güncellemelerini olay yayınlarıyla (event-bus) bütüne yayan bir yapı benimsemiştir.
2. Technological Context	Proje, dış bir API'den (FPL) alınan verilerin işlenmesi ve oyunculara çeşitli kurallar çerçevesinde puan atanması gibi karmaşık gereksinimlere sahiptir. Bu gereksinimler doğrultusunda seçilen tasarım örüntüleri (Design Patterns): Event-Driven Mimari , Puan hesaplamaları için Strategy Pattern , Bildirim senaryoları için Observer Pattern ve tekil tablo erişimi için Singleton Pattern 'dir.
3. Summary of the Method	Yöntem, dış kaynak modülünün (<code>fpl_fetcher.py</code>) belirli aralıklarla yeni istatistikleri çekerek merkezi bir mesaj veri yolu (<code>EventBus</code>) üzerinden bir olay yayımlamasıyla (<code>GAMEWEEK_DATA_FETCHED</code>) başlar. Dinleyiciler bu güncel verileri alır ve her oyuncunun pozisyonuna özgü ayrı hesaplama stratejileriyle (<code>ScoringStrategy</code>) puanlama yapar. Hesaplanmış güncel takım puanları, tüm hesapların tek noktadan eriştiği <code>GlobalLeaderboard</code> (Singleton) objesi üzerinde güncellenir.

Section	Evaluation
4. System Evaluation	Bir üniversite projesi ölçeğinde değerlendirildiğinde; mimari, bileşenlerine göre sınıflandırılmış ve roller temiz bir biçimde ayrılmıştır. Örüntülerin (Pattern) kendi içerisindeki iletişim akışı ve sorumluluk dağılımları nettir. Strategy yapısının if-else yığınlarını engellemesi ve EventBus'ın bağımlılıkları koparması, sistemin iyi bir yazılım mühendisliği algısıyla modellendiğini göstermektedir.
5. Assumption Validity	Geçerli olan varsayımlar: Liderlik tablosunun sistemde herkes tarafından erişilen tekil bir kaynak (Singleton) olması ve pozisyonlara özgü puan hesaplamalarının farklı kurallara sahip olması (Strategy) gerektiği mimari açıdan geçerlidir. Zayıf kalan varsayımlar: Gerçek dünya ölçeğinde dış API üzerinden her seferinde istisnasız bütün veri ağacının kopyalanabileceği varsayımı ve asenkron/çoklu erişimlerde Leaderboard'un Thread-Safe yapı olmadan çökmeden çalışacağı varsayımı zayıftır; ancak bir akademik proje için kabul edilebilir bir soyutlamadır.
6. Identified Weaknesses	Tasarım yöntemi en büyük zorluğu "performans maliyeti ve eşzamanlılık" konularında yaşamaktadır. FPL API'sinden gelen tüm oyuncu verilerinin fark (Delta) gözetmeksizin sürekli bir döngüde yeniden çekilmesi sistemi yavaşlatmaktadır. Ayrıca, Liderlik tablosundaki bellek (cache) güncellemelerinin Thread-Safe koruması olmaması, aynı anda puanların güncellenmesi sekanslarında geçici kilitlenmelere ("Race Condition") sebep olabilir.
7. Missing Mechanisms	1. Leaderboard'un Singleton yapısına eş zamanlı çoklu erişimleri güvenli kılması

Section	Evaluation
	adına "Thread-Safety" eksiktir (Örn: <code>threading.Lock()</code> kullanımı). 2. Takım oluşturulduğunda lig dahilindeki diğer hesapları ve UI bileşenlerini uyaracak olan Observer bildirim yapısı sadece kâğıt üzerinde planlanmış, tam kurgulanmamıştır.
8. Scenario Evaluation	Senaryo: Hafta sonu arka arkaya FPL verisi güncellendiğinde, veri çekim modülü API'ye yoğun istekler oluşturacak ve ardından EventBus üzerinden birbiri ardına hesaplama olayları <code>GlobalLeaderboard</code> üzerine akın edecektir. Eğer birçok kullanıcı aynı anda veri üzerinden puan hesaplamasına düşerse, Thread-Safe yapısı kurulmamış tablo üzerinde veri bütünlüğü bozulabilir veya en son yazan (last-write) verisi dışındaki okumalar geride kalabilir. Ligdeki kullanıcılara anlık UI bildirimi sağlayan bir Observer olmadığı için güncellemeler sessiz gerçekleşir.
9. Improvement Assessment	Önerilen çözümler, proje mevcut yapısını baştan yazmayı gerektirmeyecek kadar kolaydır ve fazlasıyla mantıklıdır. <code>GlobalLeaderboard</code> yapısına basit bir "Lock" (kilit) entegre edilmesi olası tüm çakışma (Race Condition) problemlerini çözer. Bildirim (Notification) Observer sınıfının eklenmesi ise, mevcut EventBus mekanizması üzerine kolaylıkla inşa edilebilir ve son kullanıcı deneyimini doğrudan artırır.
10. Implementation Assessment	Ortaya konulan prototip, kavramsal olarak anlatılan mimari konseptlerin yazılıma başarıyla döküldüğünü çok net göstermektedir. Özellikle farklı puanlama senaryolarının (<code>GoalkeeperScoringStrategy</code>, <code>MidfielderScoringStrategy</code>), ana iş mantığından soyutlanması geliştirici ekibin tasarım örüntülerini ve SOLID kavramlarını

Section	Evaluation
	temel düzeyde başarılı şekilde uyguladığını gösterir.
11. Overall Strengths	- Nesne yönelimli tasarım örüntülerinin (design patterns) ve SOLID prensiplerinin bağlama tamamen uygun şekilde projelendirilmesi. - `EventBus` kullanılarak sınıfların/komponentlerin birbirine sıkıca bağlanmasının önlenmesi (loose coupling). - Karmaşık oyun kurallarının yönetilebilirlik açısından mükemmel soyutlanması.
12. Overall Limitations	- Dış API kaynaklarına tam veri güncellemesi üzerinden bağımlı olunması (Optimizasyon / Caching eksikliği). - Thread-Safe yapı eksikliğinin projeyi teorik olarak gerçek dünya eşzamanlılığına kapalı bırakması. - Lig bazlı (diğer üyeleri uyarma vb.) planlanmış bildirim yapılarının eksik uygulanmış olması.
13. Recommendation	- Global Liderlik Tablosunun veri ekleme metotlarına bir asenkron mimari veya <code>threading.Lock()</code> ile güvenli yazma sağlanabilir. - Performans iyileştirmesi için API'den yalnızca en son "Event" veya Timestamp üzerinden veri getirecek bir yapı tasarlanabilir. - Takım içi güncellemelerde UI tarafına uyarı verebilecek kısmi oluşturulmuş Observer taslak yapısı tamamlanmalıdır.

Tasarımı kabul eder misiniz: ☒ **Accept** ☐ Weak Accept ☐ Borderline ☐ Weak Reject ☐ Reject

Section	Evaluation
14. Justification	Yapılan proje bir üniversite ölçeğinde değerlendirildiğinde ekip; yazılım geliştirme

Section	Evaluation
	sürecinin en kritik zorluklarından olan "doğru mimari seçimi" yapma (Strategy, Singleton, Observer, Event-driven) hedefini başarıyla kavramış ve yetkin bir örnek kod tabanı ile sunmuştur. Gerçek dünyaya ait rate limit, API optimizasyonu, ya da race condition gibi konularda eksikler barındırsa da, temel mühendislik ilkelerinin ("Design Patterns" kurgularının anlaşılması) layıkıyla yerine getirildiği açıkça görüldüğünden dolayı tasarım ve proje kabul (Accept) edilmeye uygundur.